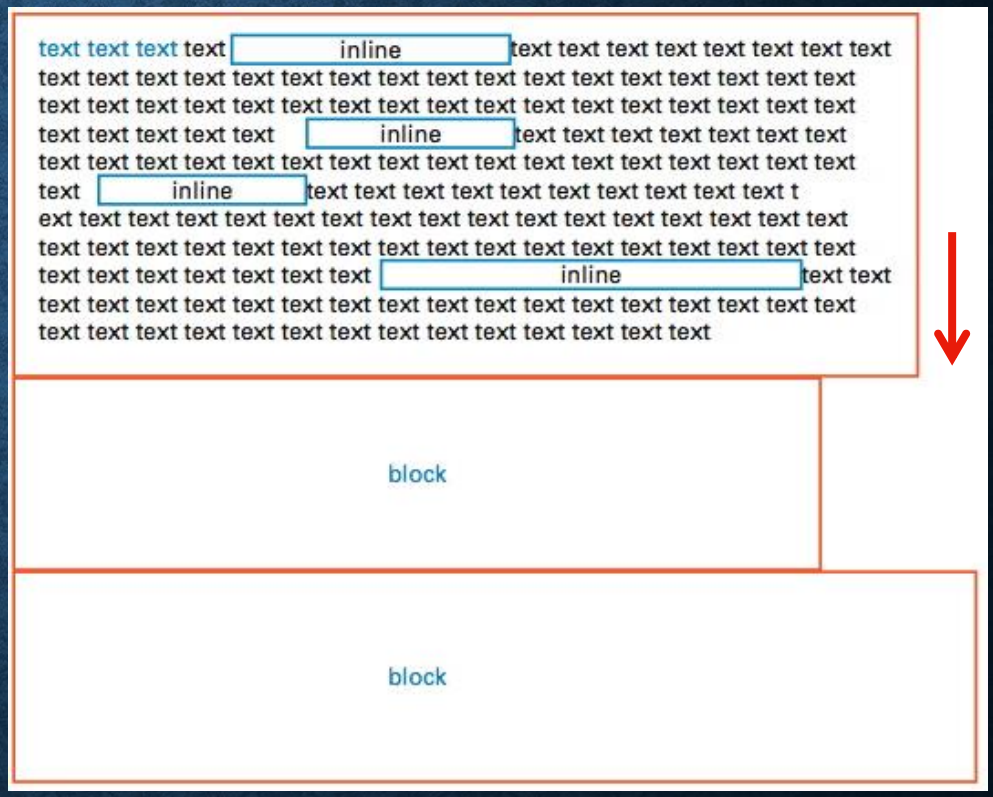
WEB TECHNOLOGY LECTURE – 05 (CSS LAYOUT)

CONTENT

- CSS Floats
- CSS Flexbox
- CSS Positioning

THE NORMAL FLOW

- CSS boxes for block elements are stacked, one above the other, so that they're read from top to bottom.
- boxes for inline elements are placed next to each other
- In CSS, this is said to be the "normal flow".



But…

Suppose we want to create following layout

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Phasellus imperdiet, nulla et dictum interdum, nisi lorem egestas odio, vitae scelerisque enim ligula venenatis dolor. Maecenas nisl est, ultrices nec congue eget, auctor vitae massa. Fusce luctus vestibulum augue ut aliquet. Mauris ante ligula, facilisis sed ornare eu, lobortis in odio. Praesent convallis urna a lacus interdum ut hendrerit risus congue. Nunc sagittis dictum nisi, sed ullamcorper ipsum dignissim ac. In at libero sed nunc venenatis imperdiet sed ornare turpis. Donec vitae dui eget tellus gravida venenatis. Integer fringilla congue eros non fermentum. Sed dapibus pulvinar nibh tempor porta. Cras ac leo purus. Mauris quis diam velit.

CSS Floats

FLOATS: POSITIONING CSS BOXES

- …we can position block element boxes side-by-side in CSS using floats.
- Setting the float property to left or right causes a box to be ***taken out of its position in the normal flow*** and moved as far left or right as possible.

FLOAT VALUES

- The Float property has three value options:
  - float: left;
  - float: right;
  - float: none;

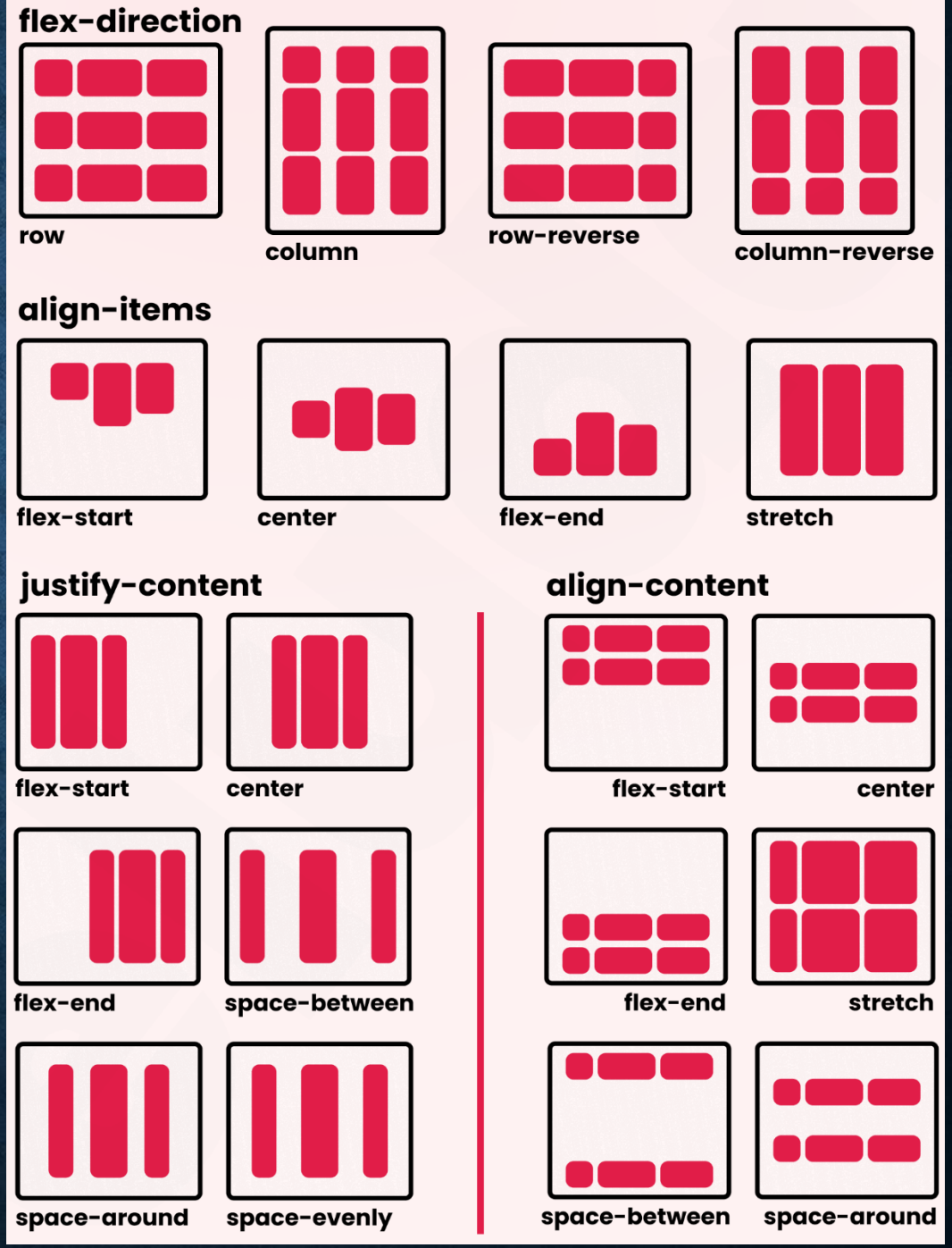
RESTORING THE NORMAL FLOW: "CLEAR"

- Floating elements can cause layout issues, so we use the clear property to prevent elements from wrapping around a floated element.
- clear: left;
- clear: right;
- clear: both;
- These specify which side of the element's box may ***not*** have a float next to it.

CSS Flexbox

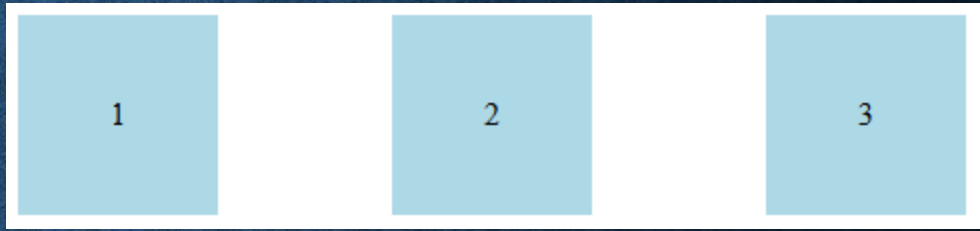
FLEXBOX (FLEXIBLE BOX LAYOUT)

- The **Flexbox** layout is a CSS module designed to create one-dimensional layout with ease.
- To enable Flexbox, apply display: flex; to a container:



FLEXBOX (EXAMPLE)

```
.container {
 display: flex;
 justify-content: space-between;
 align-items: center;
 flex-wrap: wrap;}
.item {
 width: 100px;
 height: 100px;
 background: lightblue;
 margin: 5px; display: flex;
 justify-content: center;
 align-items: center; }
```



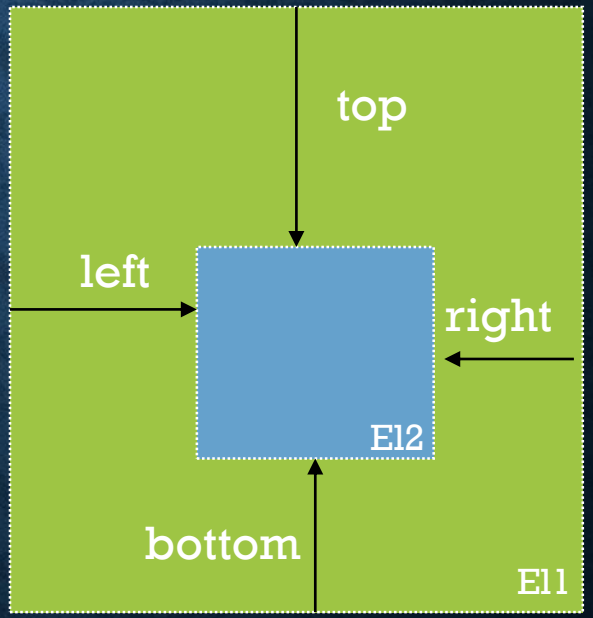

```
<div class="container">
  <div class="item">1</div>
  <div class="item">2</div>
  <div class="item">3</div>
</div>
```

CSS POSITIONING

CSS POSITIONING

- The last core concept to understand in CSS layout is positioning.
- There are five types of positioning that can be applied to CSS boxes:

1. **Static Positioning**
2. **Fixed Positioning**
3. **Relative Positioning**
4. **Absolute Positioning**
5. **Sticky Positioning**



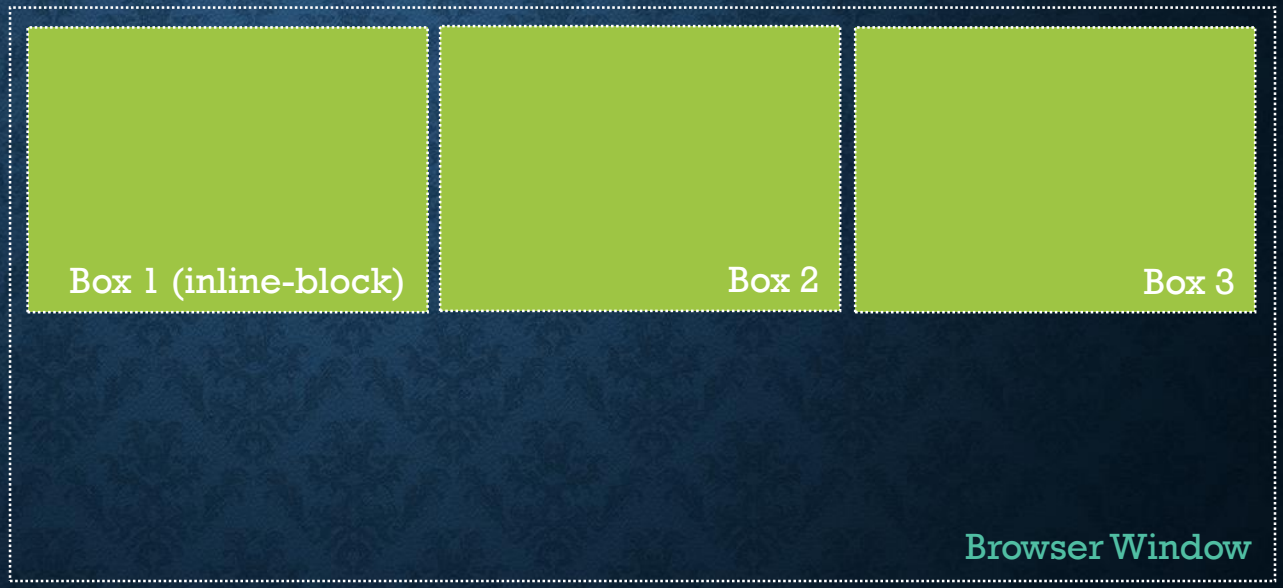
* top, bottom, left, and right are four properties related to positioning. These properties require a reference which vary with different type of positioning

STATIC POSITIONING

- HTML elements are positioned static by default.
- A static positioned element is always positioned according to the normal flow of the page.
- Static positioned elements are not affected by the top, bottom, left, and right properties.

```html
<div class="container">
    <div class="box1">Box1</div>
    <div class="box2">Box2</div>
    <div class="box3">Box3</div>
</div>
```

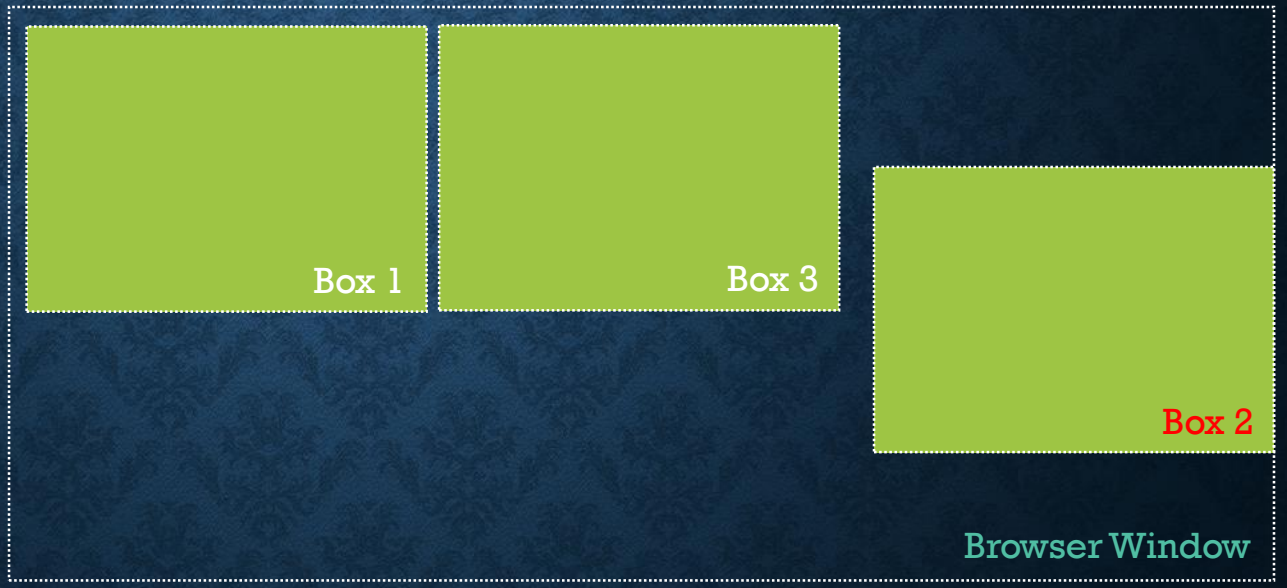
```css
.box1, .box2, .box3{
display: inline-block;
padding: 10px;
}
```



FIXED POSITIONING

- An element with fixed position is positioned relative to the browser window.
- It will not move even if the window is scrolled.

```
#box2 {
    position: fixed;
    right: 0px;
    top: 20px;
}
```

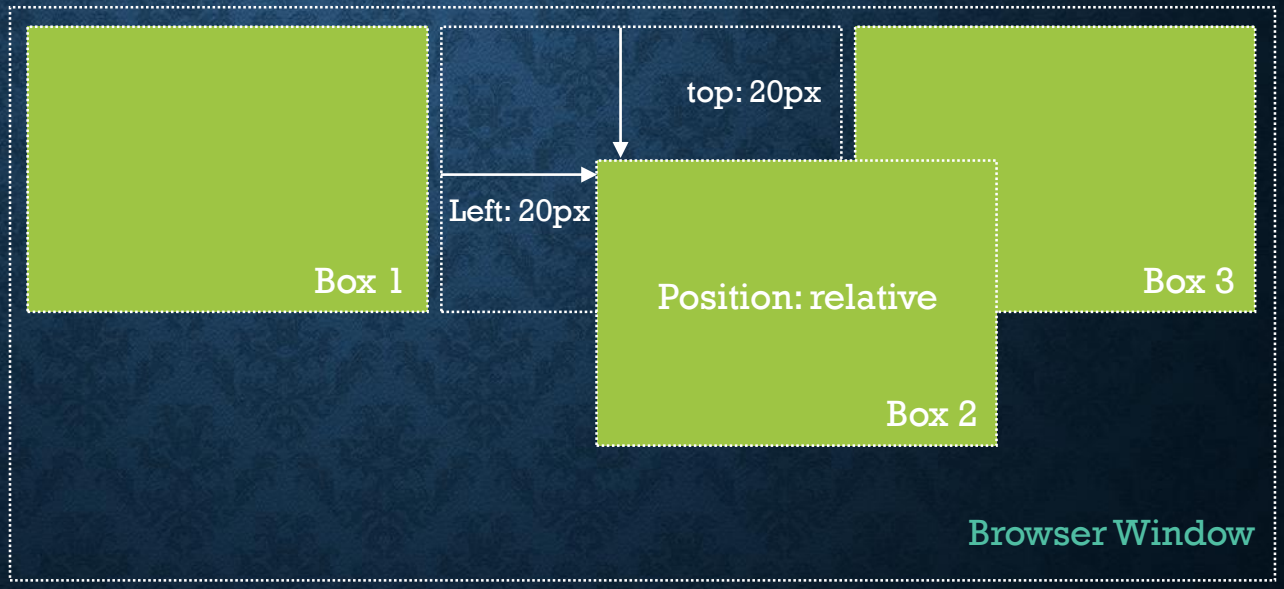


Notice that box3 has occupied the box2's normal flow position.

RELATIVE POSITIONING

- A relatively positioned element will position in relation to the normal flow.
- In this example, **box 2** is offset 20px, top and left. The result is the box is offset 20px from its original position in the normal flow. Box 2 may overlap other boxes in the flow, but other boxes still stay its original position in the flow.

```
#myBox {
    position: relative;
    left: 20px;
    top: 20px;
}
```

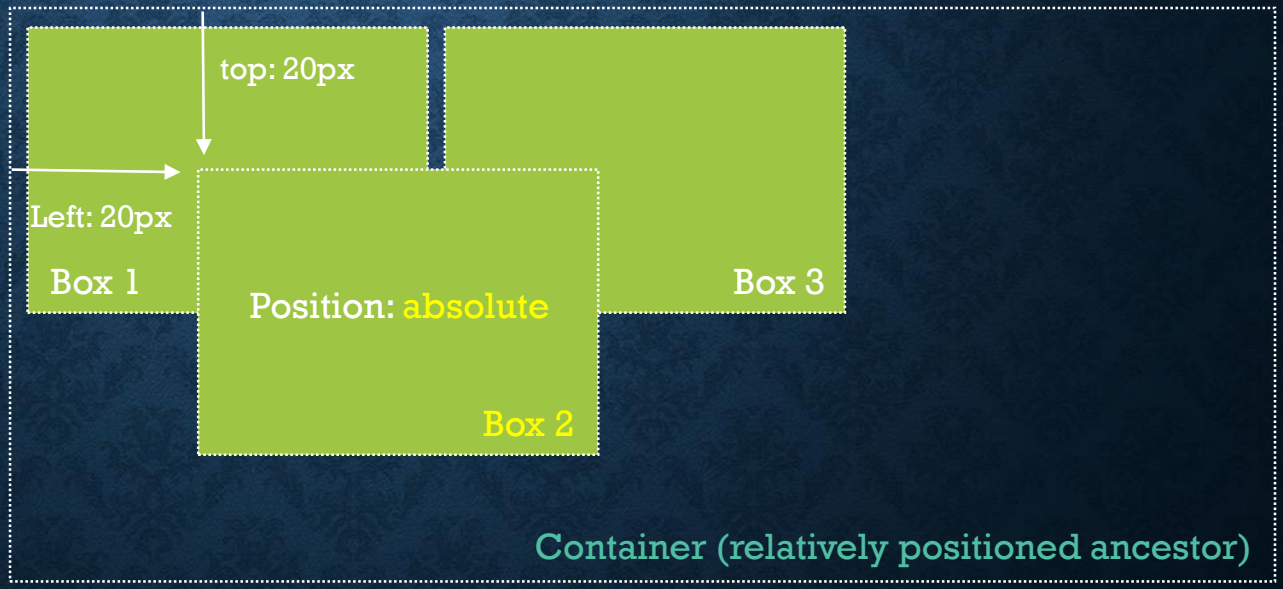


Notice that box3 does not occupy the box2's normal flow position.

ABSOLUTE POSITIONING

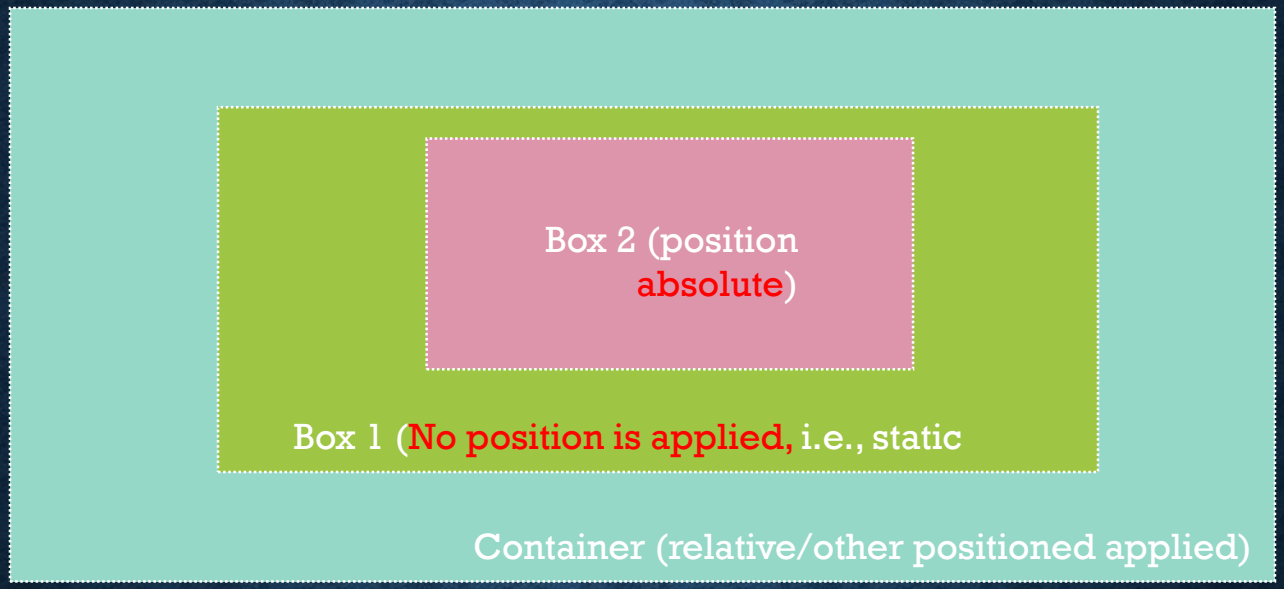
- An absolutely positioned box is taken out of the normal flow, and positioned in relation to its nearest positioned ancestor (i.e. its containing box).
- An element is called a "positioned element" when it has any value of the position property except static.
- If there is no ancestor box, it will be positioned in relation to the initial containing block, usually the browser window.

```
#box2 {
    position: absolute;
    left: 20px;
    top: 20px;
}
```



Notice that box3 has occupied the box2's normal flow position.

MORE ABOUT ABSOLUTE POSITIONING

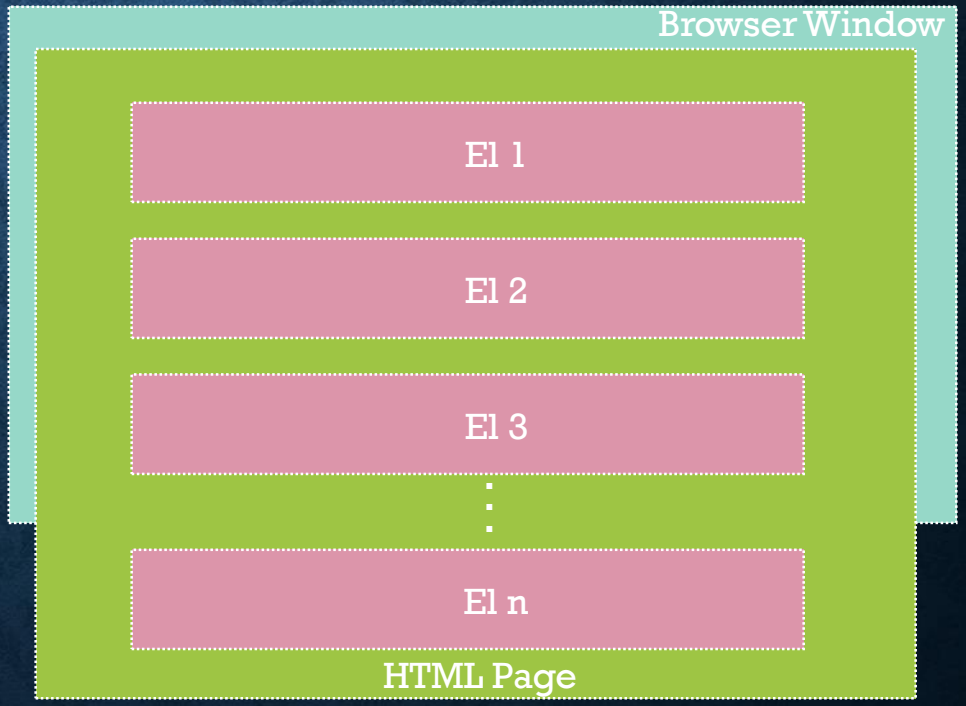


- Although box1 is ancestor of box2, the top/left/right/bottom property will be affected by the container (not by the box1)
- This happens because box1 is not a positioned ancestor.

STICKY POSITIONING

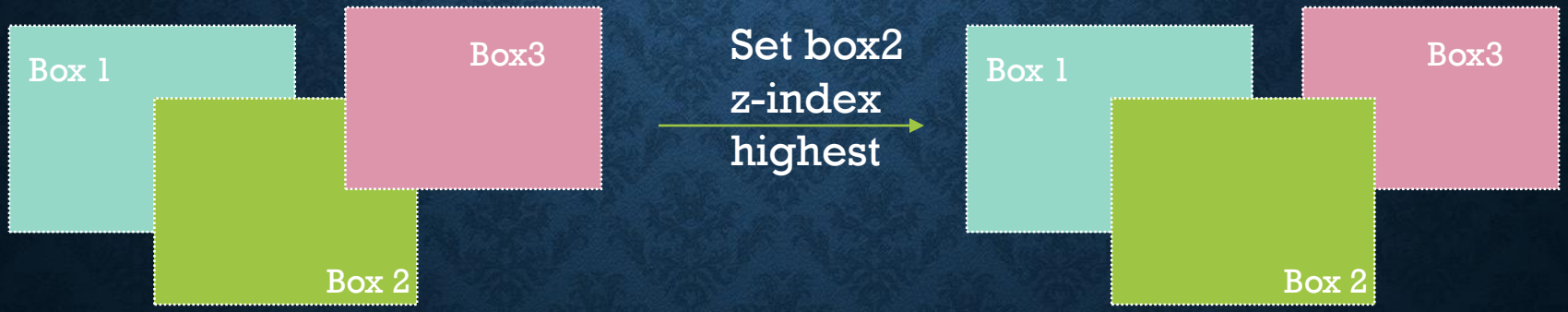
- It is used to prevent an element from getting out of window.
- Its top/left/right/bottom property activated only when it receive threads on its existence.

For example, If you to prevent El2 from being disappeared while scrolling down. Then you can set its position sticky and top to 0px;



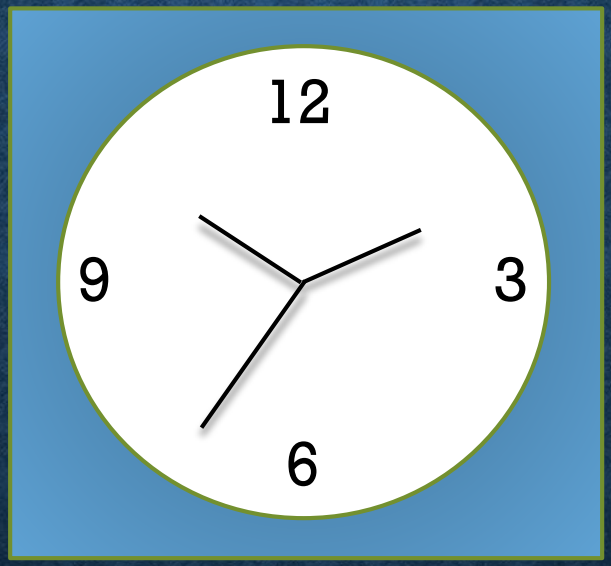
OVERLAPPING ELEMENTS

- When elements are positioned outside the normal flow, they can overlap other elements.
- The z-index property specifies the stack order of an element.



TASK

Can You Re-create the following design?



End of CSS

All slides are only for intended audience. Without consent sharing to others is prohibited.